

Lec 39 Pix2Pix GAN

Lec 39 Pix2Pix GAN

Lecture	39 / 63 + intro
Duration	34:20
Captions	English (auto-generated)
Words	4,671
Video	xS5PlsvAOB8
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone. Welcome back to the lecture series on fundamentals of generative AI and large language models. In today's session, I will be discussing about image to image translation using Pix2Pix GAN. So, we'll have a look into the contents. Introduction to Pix2Pix GAN. Difference between conditional GAN and Pix2Pix. Application of Pix2Pix GAN. U-Net architecture for generative model generator model in Pix2Pix. Patch GAN for discriminator model.

[00:36] Weight update of generator and discriminator in Pix2Pix. And training and inference of Pix2Pix GAN. So, we'll start with introduction to Pix2Pix GAN. What exactly it is. So, basically Pix2Pix is a specialized implementation of conditional GAN. So, it itself is a conditional GAN, but it is specialized for image to image translation. And it was introduced in the paper image to image translation with conditional adversarial network in the in the in this paper image to image translation with conditional adversarial network by Phillips Isola and Isola and his team from Berkeley AI research lab.

[01:27] Okay. So, we will see what is the core idea behind this Pix2Pix GAN. So, basically it is for image to image translation. That means we have one kind of image

in the input and some other kind of image as the output. So, here we can see the input is edge the sketch kind of thing and photo is the output. Okay. So, this is one kind of image to image translation. Then we have map input and satellite output. That is another one.

[02:04] Then we have a sketch input and the painting output. That is another kind of image translation. Low resolution input to super resolution output. So this is one more image to image translation. And then we have This is an image which you know, damaged. From a damaged image, it has been repaired and we have the good image from that damaged image. Okay. So that reconstruction have been done. So it is also image to image translation work.

[02:44] Now Pix2Pix is specialized for these kind of tasks where one image is the input, another image is the output. Now what is the difference between conditional GANs or cGANs and this Pix2Pix? Pix2Pix basically is a type of conditional GAN. It is a specialized conditional GAN. Now what is the difference between normal condition from the conditional GAN GAN, why it is specialized, where exactly is the difference, we will try to see that one.

[03:15] So conditional GAN is a generic generic conditional generation framework. Okay. So it is very generic. Can use labels, text, images etc. as the condition. So it is a generalized framework. Whereas Pix2Pix is a specific type of conditional GAN which is specifically designed for paired image to image translation. Okay. So this Pix2Pix has little bit of certain changes in the architecture which helps it for a image to image translation task, okay?

[03:55] Now, when the paper came out, the original paper of Pix2Pix, there were, you know, lot of artists and other, um, you know, people from the artist background or graphic practitioners, visual artists, they they used Pix2Pix in a various, uh, ways. So, these are a few of the images. The image has been taken from the original paper. So, it has been used for various applications by online community. So, it is from a sketch to, uh, say, an uh, image, and then background removal. You can see here the picture is having a background, here no background. Then the sketch to, you know, Pokémon. Then palette generation. If you see this portion is black where here that portion, uh, is having some colors. And then

[04:56] there is do as I do as I do. So, do as I do is basically a pose transfer. So, that pose is transferred to the output image. And then sketch to portrait. Then photo generator. So many various kinds of applications of this Pix2Pix GAN have been, um, ex- lot of visual artists and graphic practitioners, they have used for various kind of this applications, okay? So, there is a huge range of things that can be done using the Pix2Pix.

[05:30] Now, what is the architectural difference between conditional GAN and Pix2Pix? Let us have a look. So, this is the picture which we all know, and picture

of conditional GAN. And this is the picture for Pix2Pix architecture. If you see, the main difference, everywhere it is same. The main difference is in the generator here we had a neural network, where here it is an unit. So, a unit is used as the generator in case of pix2pix, and here the discriminator in conditional GAN was also a neural network, in this case it is a patch GAN.

[06:09] So, we will come to what a patch GAN is a little later, but for time being we will know that in pix2pix the generator is made up of a unit architecture, and the discriminator is made up of patch GAN. So, that is the main difference architectural difference between conditional GAN and pix2pix, okay? So, these are the two things that we have to concentrate on. We will have a look. So, we all know what a unit architecture looks like, and what what is the use of unit architecture? Till now, we have seen that unit is used for semantic segmentation, but now we will see how it can be used for the generator in a pix2pix. So, if we see this is a very known picture of unit, and then we have this down sampling and then bottle

[07:03] bottle neck, then there is the up convolutions, you know, the So, this is basically the encoder part, which there are operations like convolution, 3 cross 3 convolution, ReLU. So, these are convolution and ReLU, then we have max pooling in these red arrows, which shows and these blue arrows shows a 3 cross 3 convolution and ReLU, and there is a special dimension decrease with each level of max pooling, and Uh, this is this portion where the spatial dimensions are coming down. We call it as an encoder path and where the spatial here it is up convolutions. This portion is called as a decoder path or the expanding path.

[07:56] So, encoder and decoder we have. Then we have the bottleneck which is this portion and it has the compressed information, the the important information in the most compressed way. Okay? So, in this case the unit architecture just takes the input image. Suppose this is a sketch and the output image or it generates an image with the like a photo. Okay? So, the unit is basically trained for filling up, you know, which are the places which should be filled with color, what kind of color, for what kind of structure.

[08:32] So, all these are trained in the unit during the training time. Okay? And the encoder basically extract the features from this sketch. So, all the fine details, the edges, the lines, the everything, the finer details are extracted using the encoder and then the bottleneck compresses all the informations and give all the important information in the most compressed form and the decon decoder reconstructs the images. Basically, it generates the image. Generates the photo or the painting that we want to get out of the generator of the pix2pix.

[09:14] Now, these skip These are the skip connections which is a very important part and this is the reason we don't use normal encoder-decoder architecture. We pix2pix uses this unit architecture because of the presence of the skip connections

in unit. So, what happens is skip connection helps to preserve the low-level spatial details lost during the encoding. So, as we know, during the encoding, as there is a decrease in the spatial dimension, what happens is a lot of finer details start getting lost. So, that uh those information which was lost because of the skip connection between the encoder and the decoder, those lost informations are again, you know, retrieved back and placed here using the

[10:06] copy and crop and you know, crop copy and then they got go from the encoder to the decoder. Okay, so this skip connection is very important for pix2pix because otherwise, the photo that is generated or the generated output image in this case, this photo from this sketch, they will lose the finer details, okay? Now, that is about the unit that is used as the generator in pix2pix. Now, what about the discriminator in pix2pix? We have uh seen that PatchGAN is used as a discriminator in pix2pix.

[10:48] What is this PatchGAN? It's a type of discriminator that classifies whether each n cross n patch is real or fake. That means, previously we have seen in conditional GAN that the discriminator sees the entire image and then tells whether it is real or fake. In this case, what happens is if you have 256 by 256 one image, then it will patch by patch the discriminator will see whether that patch is real or fake. So, we let us have a look.

[11:24] So, suppose this is my real image and this is my generated image, okay? So, you can see the real image is having much better resolution. the generated image is very blurry, but it is capturing some of the details of the you know, uh the real image, but it is very blurry. The resolution is bad. Now, in patch GAN, what happens is these first this patch the discriminator will see, and then it will see whether this portion is exactly matching with this.

[11:53] Is it real or fake? Then another patch, then another patch, another patch, another patch. So, you know, you have patch by patch in cross in sized patch by patch the discriminator will see, and it will tell whether it is a real image or that patch resembles a real image completely or it looks like a fake image, okay? So, that is the main idea of patch GAN, and the final discriminator output is the average of all the patch decisions. So, every patch there will be a decision, right? So, whether this is real or fake, real or fake, real or fake. And the overall decision of the discriminator is the average of all these patch decisions.

[12:39] Now, the next question comes, how big should be the patch? What is the ideal um size of the patch, okay? So, let us see that one. What is the answer to that? So, in the original paper experiments have been done with various patch sizes. So, people have or the scientists working on that paper, they have done experiments. For one cross one patch size, you can see the color correctness was there, but there was no effect on the special statistics. So, you can see it is not a very clear thing. The color is um correct. The color has been uh captured correctly, but special statistics

or special information is not that clear in this one cross one patch.

[13:26] In 16 cross 16, we see there are locally sharp results. So, it is not as bloody as 1-x-1, but creates tiling artifacts. If we see these there are some artifacts here. So, there is artifacts which comes for the 16-x-16 patch size. Then, scientists have tried 70-x-70. This is having sharp output even if the if incorrect in spatial as well as spectral dimension. Means, it is having sharp output some of the places the spectral and the spatial you know, information is not exactly correct.

[14:08] And then they tried with 286 by 286 size. It is the output is visually similar to 70-x-70 patch GAN, but the metrics gives lower score. So, from the paper, it is decided since 70-x-70 patch GAN gave the best visual uh representation with the metrics performance metrics FCN is, you know, it's it's good. So, because 286-x-286 this metric went down. FCN metric score value it went down. So, this is the best trade-off for the uh patch GAN discriminator 70-x-70 patch size, okay?

[14:56] Now, we will have a look how the training of the pix2pix GAN happens. Suppose I have a sketch of a scenery and I want a painting of the same sketch, which is my target image. So, the sketch of the scenery is A and the painting is sketch B, okay? So, this is my scenery. This is the sketch. This we call as a A. And this is the target which is the painting and this is my real B. Okay, so real A and real B. Now we will look into how the training happens in the architecture. So we have the generator, basically it's a unit model and then the discriminator which is the patch GAN. We have already uh discussed about it. Then the real image and the real uh real A and real B.

[15:48] plus So both of them go to the patch GAN. Both goes to the discriminator, okay? So the discriminator's input is real A as well as real B. Whereas the generator takes real A as the input, okay? Now it generates a fake B. So it generates some image, if you can see the resolution, it generates something but it is not as same as uh same resolution as real B. So this is a fake B. And then it adds this real A and both of these things go to the discriminator, okay?

[16:28] So the discriminator gets real A, real B fake B as well as the real A, okay? So it has information of all the things here. Then the discriminator you know, sees whether it is real or decides whether it is real or fake. As we know, the discriminator gives a probability score whether it is real or fake. And depending on the correctness of the answer there is discriminator loss and generator loss. These are back propagated, the discriminator loss gets back propagated to the discriminator and fine-tuning happens on the process of training the discriminator, whereas generator loss gets back propagated to the generator and through iterations of training the generator becomes more and more um efficient in making

[17:20] fake images look very similar to real images. Okay? So, this is what happens in the architecture of pix2pix. And this loss is basically the GAN loss or the binary cross entropy loss. We will look into detail in details what exactly it is in the subse-

quent this ones in the next slides. Okay? Now, we will have a look how the weight updation happens in the generator during the training. We know, as I was telling, in each and every iteration, the generator becomes better and better in its performance that it starts generating images, fake images, which looks very similar to the uh real image. Okay? Now, for doing this one, that there should be some weight updation that is required in the generator. That is That is how the

[18:11] performance can be improved. Okay? Now, we will see how the weight updation happens in the generator. So, we know the generator is a unit and the input image to the generator is the sketch, which is your real sketch, and it outputs some fake B, okay? Output some generated image of a painting. Now, and this is your target image, which is the actual painting. Okay? Which is your real B. Now, what happens is whatever is taken out of the generator or whatever is generated by the generator, and whatever is the target image, these two are processed through mean absolute error loss. So, what exactly it means?

[19:01] It means that this mean absolute error, it takes out the difference between the actual target and the generated target. Okay? So, pixel by pixel, how much it is varying, that is captured by the mean absolute error loss, which we'll call as the L1 error loss. Now, so, this is one part of the error. Now, this generated image by the generator goes to the discriminator, and all labels are one. I will come to this one. I will explain a little later.

[19:36] We'll first see the entire process, and then I will come to why I have written this all labels are one. Okay? So, once the generated image goes to the discriminator, then the discriminator also gets the target image here. Okay? So, it goes to the target the target image goes to the discriminator also, and then what happens is During training, the discriminator gets the real target image as well, and then it is able to distinguish between whether the generated image looks similar to the actual target, you know, image.

[20:17] So, it will find out the binary cross entropy loss between the generated image and the target image. And here we have seen that we have taken a mean absolute error loss. What was that absolute error loss? Between pic pixel by pixel, there was the difference between the generated image and the target image. Now, that mean absolute error gets multiplied by a factor lambda, which is in the original paper they have taken it as 100. We'll come to that, uh, you know, later. Uh, I will speak a little more about lambda, why 100 they have taken.

[20:59] And this lambda into the mean absolute error loss and the binary cross entropy loss, these two are added together and we get whatever we get as a total loss, that is passed through the gradient descent. So, the gradients are applied and the gradient descent happens. The loss for minimizing the loss, it goes back to the generator. Again, the generator's weight updation happens and the training process continues.

Okay? So, this is the overall how the weight updation happens in generator, what exactly happens in each and every phase we have have we had a look. And now I'm coming to all labels are one.

[21:47] So, what exactly I mean by this statement? See, you during the generator's training, fake images are passed through the discriminator. We have seen this is the fake image. So, this is the my fake image. Correct? Now, this fake image is passed through the discriminator. Now, the generator wants the discriminator to predict this one as one. This is D as we all know, this is the discriminator's score probability score for this real image and the generated image. And the generator wants it to be one because it wants to fool the discriminator.

[22:28] So, it is labeling all the fake images as one. So, the binary cross entropy compares fake outputs against the label one. So, it is label one is there and the fake images are there and then the binary cross entropy happens here, okay? So, that is the reason all labels are one I have I have kept it here, okay? So, the total generator loss is the binary cross entropy loss plus λ L1 loss, okay? Now, this λ value has been taken as 100 here.

[23:03] To give more importance to pixel by pixel difference between the fake image and the uh actual target image. Now, we will see what is a generator loss in pix2pix GAN. So, we'll have the mathematical things here. So, total loss is the cGAN loss, conditional GAN GAN loss, plus λ into L1 loss. Now, the cGAN loss is basically the adversarial loss or the conditioned GAN loss. We all know that one. Uh just to give uh to make the mathematical expressions a little more um easy to understand, I have put the real image original image as X here. And uh real target image as Y here. And the generated target image as G of GX . So, it is generated by the generator. What it is trying to generate is X , okay?

[23:59] Now, what is the mathematical form for the uh cGAN loss or the conditional GAN loss. Conditional GAN loss is written as the expectation value $\log$ expectation value of $X, Y \log D(X, Y)$ plus expectation value $X, Y \log 1 - D(X, Y)$ and GX . So, what exactly they are? We all know this is the um This is the uh probability score given by the discriminator for the real image. And uh real real pair of images, the original and the target image. Whereas, this is the real image and the generated target image, okay? And if you see uh this is the cGAN loss. And the L1 loss is basically it is the difference between your actual target and the target that is made or generated using the generator. So, it's an absolute

[25:08] » [snorts] » difference between them. So, it is the L1 loss. And this ensures that the generated image is pixel wise close to the target. So, pixel by pixel there will be comparison and this loss ensures that if when this loss is minimized, that means each and every pixel of the generated image is coming closer to the pixel of the target image. Okay, each and every pixel. Now, in the original paper λ is equal to 100

they have taken and this is used to strongly emphasize L1 reconstruction loss. That means more importance is given for this pixel wise comparison and reduction in the error.

[25:57] Okay. Okay. So, this gives a strong weight to pixel similarity. Okay. So, that is what is measured in the L1 loss. Now, these are some results from the original paper demonstrating the output quality for various type of losses. There may be a question why we have taken lambda this L1 loss. Why it is not only the conditional GAN loss? Why it is not only the L1 loss, but why it is a combination of both? So, the scientists working on that paper they have done experiments with So, these are the input images. These are the ground truths.

[26:39] This is with only L1 loss. This is only with conditional GAN loss. And this is L1 plus conditional GAN loss. So, if we see for with L1 the outputs are not that great. Only with conditional also, yeah, there is improvement, but if we see the combination of L1 plus C GAN, it is much better. So, that is the reason L1 lambda into L1 uh plus conditional GAN's loss is the loss that is taken for this uh architecture. Now, weight update in discriminator, we will see what exactly happens here. So, we have the discriminator, which is basically GAN.

[27:22] So, we have the patch GAN here. Okay, and the patch GAN gets the target image. And patch GAN gets the input image also. And the input image goes to the generator also. So, discriminator gets target as well as the input image. The generator gets only the input image. And if you see, the generator generates the fake image here. Okay, and this goes to the discriminator. Now, the discriminator will find will calculate two losses. One is called as the real loss, which is given by minus expectation value over $XY \log D$ of XY . Why it is real loss?

[28:10] Because we are taking the real image and real original image and real target image, okay? And there is uh you know, it is a sigmoid cross entropy between all ones and the real images. Why? Because here it is the discriminator will be giving or tagging all the real images, the original and the target images, they will be tagged as one because they are real, okay? So, uh the loss will be calculated between all ones and the real images. And here we have the generator loss, which is basically sigmoid cross entropy between all zeros and fake images. Because the generator will tag every generated image as zero.

[28:59] And so the loss is basically the sigmoid cross entropy loss between all zeros and fake images, okay? So this is expression for fake loss and this is the real loss sorry, not fake loss, generated loss and this is the real loss. And then these two are added together and we get whatever is the total loss, the gradient is applied and then gradients are because of the gradients the weight updation happens in the sorry, weight weight updation happens in the discriminator, okay? So this is how the discriminator gets trained.

[29:40] And total discriminator loss is L_{real} plus this one, which we call we can call as the generated loss also. Now we will have a look into the training details. What exactly are the parameters? So during the training of pix2pix one gradient descent step in discriminator is done and then that is fixed and then the generator training is done. So it is done discriminator, then discriminators weight updations happen, it is kept constant, then the generators weight updation happens. After that once again it goes iteratively, discriminator happens, generator is that time it is kept constant and once the discriminator is trained, that is kept constant, generators weight updation happens. So this is this happens in an iterative

[30:32] way. The optimizer that is used in the paper is an Adam solver and the learning rate is 0.0002 with beta 1 and beta 2 values of .5 and .999. Okay, so these are the parameters of the during the training process that is used. And a very important thing which I would like to mention during the inference discriminator is not used because the role of the discriminator is to push the generator to improve its performance to a level where the generator can generate images which exactly matches with the real image or very similar to the real image, okay? So, during the training time both the generator and discriminator will be there, but during the inference time discriminator is not not used. It is only the generator with

[31:30] all the you know, learned parameters that only works and it generates the images, fake images. And batch size is usually in during the inference it is one one to 10. Okay. Now, objective function of Pix2Pix, what is the objective function? We know that CycleGAN sorry, conditional GANs we have this this is the loss. Okay, and the objective function is given by $G^* = \arg\min_G \max_D L_{\text{conditional GAN}} + \lambda L_1$. So, this is what exactly it means. $\arg\min_G$ means the generator tries to minimize the objective function.

[32:21] And uh $\max_D$ means the discriminator D tries to maximize the objective objective function and this is my L_1 loss multiplied by λ , okay? So, this is um the objective function about of the pix2pix. And with this one, I come to the end of my session today. Before I conclude fully, let me summarize little bit about what we have learned today. So, we have studied pix2pix GAN, which is a specialized conditional GAN framework, which is specifically used for image-to-image translation. So, the main difference between a normal conditional GAN, which is a generalized version, it can take the conditions from images, text, or any other modalities, but here in this case, the main difference of architecture

[33:20] between the standard conditional GAN and pix2pix is that the generator used is a unit architecture, and the discriminator is a patch GAN. And that helps in the image-to-image translation. Okay. So, we have seen that architectural difference. Then, we have seen how the weight updations happen in the generator and the discriminator. What are the different kind of losses that are coming into picture during weight updation in the generator and discriminator. And you know, finally, we have

had a look into what is the objective function of the pix2pix GAN looks like. And So, with that one, we come to the end of today's session.

[34:04] In my next session, I will be discussing about CycleGAN, that is another variant of GANs. Till that time, happy learning. Thank you all. »